Hashing

Problem 1.

For each of the proposed hash functions given, determine whether they can be implemented and used as actual hash functions for a hash table with N slots. Indicate the reason.

(a) $h(k) = 0$

(b) $h(k) = \texttt{random}(0, N)$ (where $\texttt{random}(0, N)$ gives a uniformly generated random number from 0 to $N - 1$).

(c) $h(k) = k \mod N$.

(d) $h(k) = k \mod (N - 2)$ (Assume $N \geq 3$).

(e) $h(k) = f(k)$ (where $f(N)$ is a deterministic function that runs in $\Theta(N^2)$ time and returns a value from 0 to $N - 1$).

Problem 2.

Consider the hash function $h(k) = k \bmod m$, where $m = 2^p - 1$ and k is a character string interpreted in radix (base) 2^p . Show that if we can derive string x from string y by permuting its characters, then x and y hash to the same value.

Problem 3.

Comparing open addressing to traditional chaining implementations, are the following statements true under the same loading factor? Indicate the reason.

(a) In open addressing it takes on the average less time to do an unsuccessful search.

(b) In open addressing insertions and deletions are faster.

(c) In open addressing we save storage space since we don't need pointers, so we can, for the same storage cost, build larger hash tables.

(d) Collisions (i.e., that when we insert a key we don't find an empty slot) are equally probable (under same loading factor, assuming the uniform hashing assumption)

Problem 4.

(a) Consider a hash table of size $m = \sqrt{n}$. Then under the uniform hashing assumption, as $n \to \infty$, the number of empty slots tends to (i) a constant , (ii) zero, or (iii) infinity? Why?

(b) If $m = cn, c > 0$, then as $n \to \infty$, the number of empty slots tends to (i) a constant , (ii) zero, or (iii) infinity? Why?

(c) If $m = n^a, a > 1$, then as $n \to \infty$, the number of empty slots tends to (i) a constant , (ii) zero, or (iii) infinity? Why?

Problem 5.

Let H be an open address hash table, initialized with 4 slots. Assume that the load factor cannot exceed 0.75, and assume that re-hashing is done by doubling. During each insertion of a new element into H, suppose that the hash function used to generate a probe sequence is obtained via linear probing, based on the auxiliary hash function $h(k) = k \mod m$, where m equals the number of slots in the hash table. Draw the final open address hash table obtained after inserting into H the following integers $12, 43, 13, 88, 23, 26, 11, 5$ in this given order.

Problem 6.

Collision Resolution Assume simple uniform hashing in the entire problem.

(a) Consider a hash table with m slots that uses chaining for collision resolution. The table is initially empty. What is the probability that, after 4 keys are inserted, there is a chain of size exactly 3?

(b) Consider a hash table with m slots that uses open addressing with linear probing. The table is initially empty. A key k_1 is inserted into the table, followed by key k_2, and then key k_3. What is the probability that the total number of probes while inserting these keys is at least 4?

(c) Suppose you have a hash table where the load-factor a is related to the number n of elements in the table by the following formula:

$$a = 1 - \frac{1}{\sqrt{n}}$$

If you resolve collisions by open addressing, what is the expected time for an unsuccessful search in terms of n?

Amortized analysis

Problem 7.

(a) For the case table doubling and halving, using the charging method (i.e., define the amortized cost by assigning appropriate additional cost to past operations) prove that the amortized cost both of insertions and deletions is $O(1)$. Table doubling occurs when the number of stored elements n becomes $n = m$, in which case $m \to 2n$ with cost $\Theta(m)$. Table halving occurs when the table becomes $1/4$ full, i.e., $n = m/4$, in which case $m \to 2n$, incurring cost $\Theta(m)$. Hint: the case of insertions is proved in the slides. Use the same idea to prove the case of deletions.

(b) For the case of $2-3$ trees use the charging method to prove that we can use amortized costs of $O(\log(n))$ for insertion and $\emptyset$ for deletions, where n is the size of the set of keys in the tree at the time of the insertion.